

# SPI/I2C Peripheral Controller: RTL Design, Verification, and Synthesis

[Your Name]

[Your Institution / Affiliation]

**Abstract**— This report presents the design, verification, and synthesis of a dual-protocol, dual-role peripheral controller supporting both the Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) protocols in either master or slave role, selected at runtime through a memory-mapped register interface. The design was implemented in synthesizable Verilog and verified with a SystemVerilog testbench instantiating two controller instances in a master/slave configuration on shared buses. Verification surfaced five distinct functional bugs in the SPI and I2C engines, each isolated through simulation-log analysis, targeted debug instrumentation, and waveform inspection; each is documented with its symptom, root cause, and fix. The design was subsequently synthesized with a generic open-source flow to obtain structural gate-count results. The final regression achieves 14 of 14 passing checks with full coverage of the targeted SPI operating modes and representative I2C transaction types.

**Keywords**— SPI, I2C, RTL design, functional verification, SystemVerilog, logic synthesis, clock gating, digital design.

## I. INTRODUCTION

Serial Peripheral Interface (SPI) and Inter-Integrated Circuit (I2C) are two of the most widely used synchronous serial protocols in embedded and mixed-signal systems. This work presents a single peripheral controller that implements both protocols, and both bus roles (master and slave) within each protocol, with the active configuration selected at runtime rather than fixed at synthesis time.

The objectives of this project were fourfold: (i) design a synthesizable controller with a control-logic state machine implementing standard SPI and I2C timing and signaling; (ii) build a SystemVerilog testbench with self-checking scoreboards and coverage tracking to verify protocol compliance across master and slave roles; (iii) carry the design through a logic-synthesis flow to obtain structural area estimates; and (iv) apply clock gating as a low-power design technique and confirm, through simulation, that it does not compromise correctness.

The remainder of this report is organized as follows. Section II describes the system architecture. Section III details the register interface and constituent modules. Section IV describes the verification methodology. Section V, the principal contribution of this report, documents five functional bugs found and fixed during verification, together with the evidence used to isolate each one. Section VI presents synthesis results. Section VII summarizes final results, and Section VIII concludes.

## II. SYSTEM ARCHITECTURE

The controller (`spi_i2c_top`) is a single design that behaves, at any given time, as exactly one of four engines: an SPI master, an SPI slave, an I2C master, or an I2C slave. Selection is made through a register file accessed over an APB-lite interface, which also supplies the clock divider, target/own address, and data to be transmitted. Fig. 1 shows the block-level architecture.

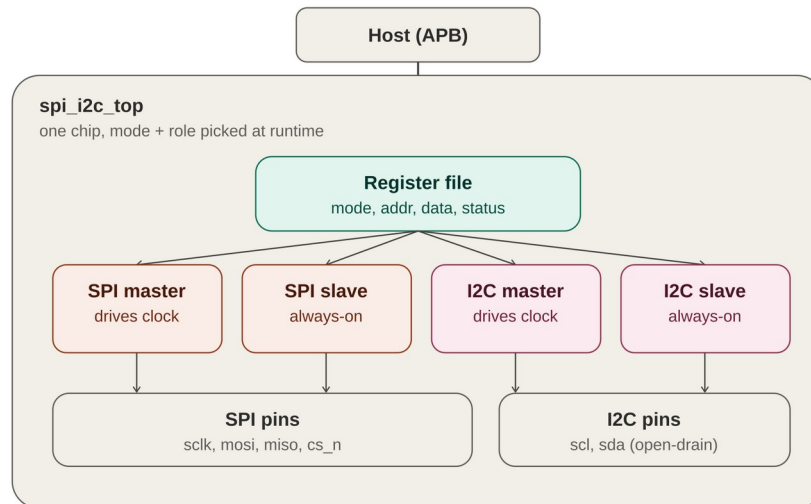


Fig. 1. Block-level architecture of `spi_i2c_top`. The register file selects exactly one of four engines; master engines are individually clock-gated, while slave engines run on the always-on system clock (see Section V-C).

A significant architectural decision, reached only after diagnosing a functional bug (Section V-C), is that clock gating is restricted to the two master engines. The slave engines each contain an input synchronizer that continuously tracks an external, asynchronous bus; gating that synchronizer's clock while its engine is deselected leaves it holding a stale bus level, which was observed to cause an intermittent, run-order-dependent verification failure. Master engines do not exhibit this issue because they drive the bus

according to their own internal timeline rather than reacting to an external one.

### III. REGISTER INTERFACE AND MODULE DESCRIPTIONS

#### A. Register map

The controller exposes a word-aligned, APB-lite register interface, summarized in Table I. All four engines share this single register set; the fields that are meaningful at any time depend on the currently selected mode and role.

TABLE I REGISTER MAP

| Offset | Name   | Fields                                                                          |
|--------|--------|---------------------------------------------------------------------------------|
| 0x00   | CTRL   | [0] enable [1] mode (0=SPI,1=I2C) [2] role (0=MASTER,1=SLAVE) [3] cpol [4] cpha |
| 0x04   | CLKDIV | [15:0] engine clock divider                                                     |
| 0x08   | TXDATA | [7:0] next byte to transmit                                                     |
| 0x0C   | RXDATA | [7:0] last byte received (read clears rx_valid)                                 |
| 0x10   | ADDR   | [6:0] I2C target address (master) / own address (slave)                         |
| 0x14   | CMD    | [0] start (self-clearing) [1] rw (0=write, 1=read)                              |
| 0x18   | STATUS | [0] busy [1] done(W1C) [2] ack_err(W1C) [3] arb_lost(W1C) [4] rx_valid          |
| 0x1C   | IRQEN  | [0] done_ie                                                                     |

#### B. Module descriptions

**clk\_gate.v:** a latch-based integrated clock-gating cell. A negedge-triggered latch re-times the enable signal so the gated clock cannot glitch. It synthesizes, in a generic flow, to exactly two cells (Section VI).

**spi\_master.v / spi\_slave.v:** full-duplex SPI engines supporting all four CPOL/CPHA modes with a programmable clock divider. Each engine uses two independent shift registers, one for transmit and one for receive, so the drive edge and sample edge — which differ depending on CPHA — never alias transmitted and received data into the same bits.

**i2c\_master.v / i2c\_slave.v:** single-byte-per-transaction I2C engines implementing START, 7-bit addressing with read/write, ACK/NACK, one data byte, and STOP. The master supports clock stretching and a simplified arbitration-loss check; the slave performs address matching and drives ACK/NACK and read data from its own synchronized view of the bus.

**spi\_i2c\_regs.v:** the register file of Table I — a zero-wait-state APB slave with self-clearing command strobes and write-one-to-clear sticky status bits.

**spi\_i2c\_top.v:** top-level integration of the register file, the two clock-gate cells, and all four protocol engines, multiplexing each engine's pins onto the shared SPI and I2C physical pin sets.

#### A. Testbench structure

The testbench instantiates the controller twice, one instance configured as master and the other as slave, sharing a common SPI bus and a common, pulled-up I2C bus. This verifies real master-slave interoperability rather than a pure self-loopback. An APB interface with write/read tasks drives each instance's register file; directed test tasks configure both instances, issue a transaction, poll for completion, and self-check the result.

#### B. Test plan

- SPI: all four CPOL/CPHA combinations, each with a full-duplex byte exchange including 0x00 and 0xFF corner-case bytes.
- I2C: a write to a matching slave address, a read from a matching slave address, and a write to a mismatched address, which must be NACKed.

#### C. Assertions

A bind-based checker file implements SystemVerilog Assertions covering engine-select mutual exclusion, SCLK stability while idle, SDA stability except during START/STOP, and absence of unknown (X) states mid-transfer. Icarus Verilog, used for the primary regression, does not implement concurrent SVA or covergroups; these checks are intended for a simulator with full assertion support, while the day-to-day regression relies on the self-checking scoreboard.

## IV. VERIFICATION METHODOLOGY

TABLE II FUNCTIONAL COVERAGE RESULTS

| Coverage goal                   | Result       |
|---------------------------------|--------------|
| SPI CPOL × CPHA modes exercised | 4 / 4 (100%) |

## V. BUGS FOUND AND FIXED DURING VERIFICATION

This section documents the principal contribution of the verification effort: five functional bugs found through simulation-log analysis and, in two cases, through added debug instrumentation and a captured waveform, rather than by code inspection alone.

### A. SPI master: incorrect branch for the final received bit

**Symptom:** the SPI master captured an incorrect byte from the slave in three of four CPOL/CPHA modes, consistently the expected byte shifted by one bit (e.g., expected 0x5A, observed 0xB4).

**Root cause:** the final-bit capture logic used a ternary expression that selected the wrong branch for CPHA=0 versus CPHA=1. For CPHA=0 the last sample had already occurred on the previous leading edge; for CPHA=1 it occurs on the very same edge as the check. The two branches were swapped.

**Fix:** the ternary branches were exchanged so CPHA=0 uses the settled shift register directly and CPHA=1 builds the final value from the live input bit.

### B. SPI slave: premature bit-shift on the first CPHA=1 edge

**Symptom:** in CPHA=1 modes only, the slave's transmitted byte was consistently shifted by one bit.

**Root cause:** the slave shifted its output register on every leading clock edge, including the first edge of a byte, which should instead hold the pre-loaded most-significant bit steady. The master already guarded against this; the slave did not.

**Fix:** the same first-edge guard used in the master was added to the slave.

### C. Clock-gated slave engines retaining stale synchronizer state

**Symptom:** I2C transactions failed intermittently, with the specific failing check varying by prior test history — evidence of state leakage in an otherwise fully deterministic design.

**Root cause:** the slave engines were clock-gated when deselected, freezing their input synchronizers mid-flight. On

re-enable, the synchronizer resumed from a stale bus level and could mis-detect the first edge of the next transaction.

**Fix:** slave engines were moved to the always-on system clock; only master engines remain clock-gated (Section II).

### D. I2C slave: SDA released before the ACK phase ended

**Symptom:** a write to a matching address was unexpectedly NACKed, and no byte was registered as received, despite correct address matching.

**Root cause:** the data-receive state released SDA unconditionally on every active cycle, including its first cycle, which was still mid-way through the preceding ACK bit's high phase. An SDA rising transition while SCL is steady-high is indistinguishable, per the I2C specification, from a STOP condition, silently aborting the transaction.

**Fix:** the release was made conditional on the falling edge of SCL, consistent with the pattern used correctly elsewhere in the same module.

### E. I2C slave: ACK line released one phase early at transaction end

**Symptom:** after fixing V-D, a write transaction completed correctly end-to-end, yet the master's status register still reported an acknowledgment error.

**Root cause:** the slave returned to idle, releasing the ACK line, as soon as it detected the rising edge of SCL for the final ACK bit. The master does not finish sampling that bit until its own, separately timed high-phase counter completes several cycles later, by which point the line had already floated high.

**Fix:** an additional holding state was introduced that keeps the ACK line driven until the master genuinely pulls SCL low again, confirmed against a captured simulation trace identifying the exact cycle of premature release.

### F. Testbench-level issues

Two issues were found in the verification environment rather than the design under test: sticky STATUS bits were not cleared between consecutive I2C tests, causing a passing transaction to be misreported due to a leftover flag; and the automated regression script initially compiled the SVA assertion file under Icarus Verilog, which cannot parse concurrent assertions, causing the automated flow to fail despite a correct design. Both were corrected in the testbench and regression script respectively.

TABLE III SYNTHESIZED CELL COUNTS (GENERIC LIBRARY)

| Module             | Cells | Notes                     |
|--------------------|-------|---------------------------|
| spi_i2c_top (glue) | 81    | Mode/role decode, pin mux |
| spi_i2c_regs       | 181   | Register file             |
| spi_master         | 231   |                           |
| spi_slave          | 106   |                           |

|               |        |                           |
|---------------|--------|---------------------------|
| i2c_master    | 295    | Largest — most FSM states |
| i2c_slave     | 175    |                           |
| clk_gate (×2) | 2 each | 1 AND + 1 flip-flop       |
| Total         | 1073   | Full hierarchy            |

## VI. SYNTHESIS RESULTS

The design was synthesized with a generic, technology-independent Yosys flow, no target cell library being available in this environment; a liberty-targeted synthesis script and an OpenSTA-based static-timing-analysis script are included for use once a process design kit is available. Table III summarizes the resulting structural cell counts.

A result worth noting: clk\_gate synthesizes to exactly two cells, an AND gate and a scan-capable flip-flop, confirming that the clock-gating idiom used is minimal and well-formed at the gate level, not merely correct in simulation.

## VII. RESULTS AND DISCUSSION

The complete test suite was executed both directly via the simulator and through an automated Python regression script that compiles, simulates, and optionally runs a Yosys structural lint pass, producing a single pass/fail verdict suitable for continuous integration. The final result is:

**PASS: 14 FAIL: 0 — REGRESSION: PASS**

The shared SPI and I2C bus signals were additionally captured to a VCD waveform and inspected, confirming chip-select assertion for the duration of an SPI byte transfer with exactly eight SCLK pulses, MOSI/MISO transitions consistent with the configured CPOL/CPHA mode, and, on the I2C bus, SDA transitions while SCL is held steady marking valid START and STOP conditions. The two protocols' pins were confirmed mutually exclusive in time, consistent with the pin-multiplexing design.

## VIII. CONCLUSION AND FUTURE WORK

This work delivered a verified, dual-protocol, dual-role peripheral controller and, more significantly, a documented record of the verification process by which five real functional bugs were located and corrected using simulation logs, targeted debug instrumentation, and waveform inspection. Natural extensions include slave-side clock stretching for I2C, multi-byte I2C bursts within a single START/STOP, a complete multi-master arbitration test, and re-running the synthesis and timing-analysis scripts against an open process design kit, such as SkyWater sky130, to obtain real area and timing figures in place of the generic gate counts reported in Section VI.

## REFERENCES

- [1] NXP Semiconductors, "I2C-bus specification and user manual," UM10204, Rev. 7.0, 2021.
- [2] Motorola Inc., "SPI block guide," V03.06, 2003.

- [3] C. Wolf, "Yosys Open SYnthesis Suite," [Online documentation], YosysHQ.
- [4] S. Williams, "Icarus Verilog," [Online documentation].
- [5] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017.